

SMART EMERGENCY PATIENT COORDINATION SYSTEM

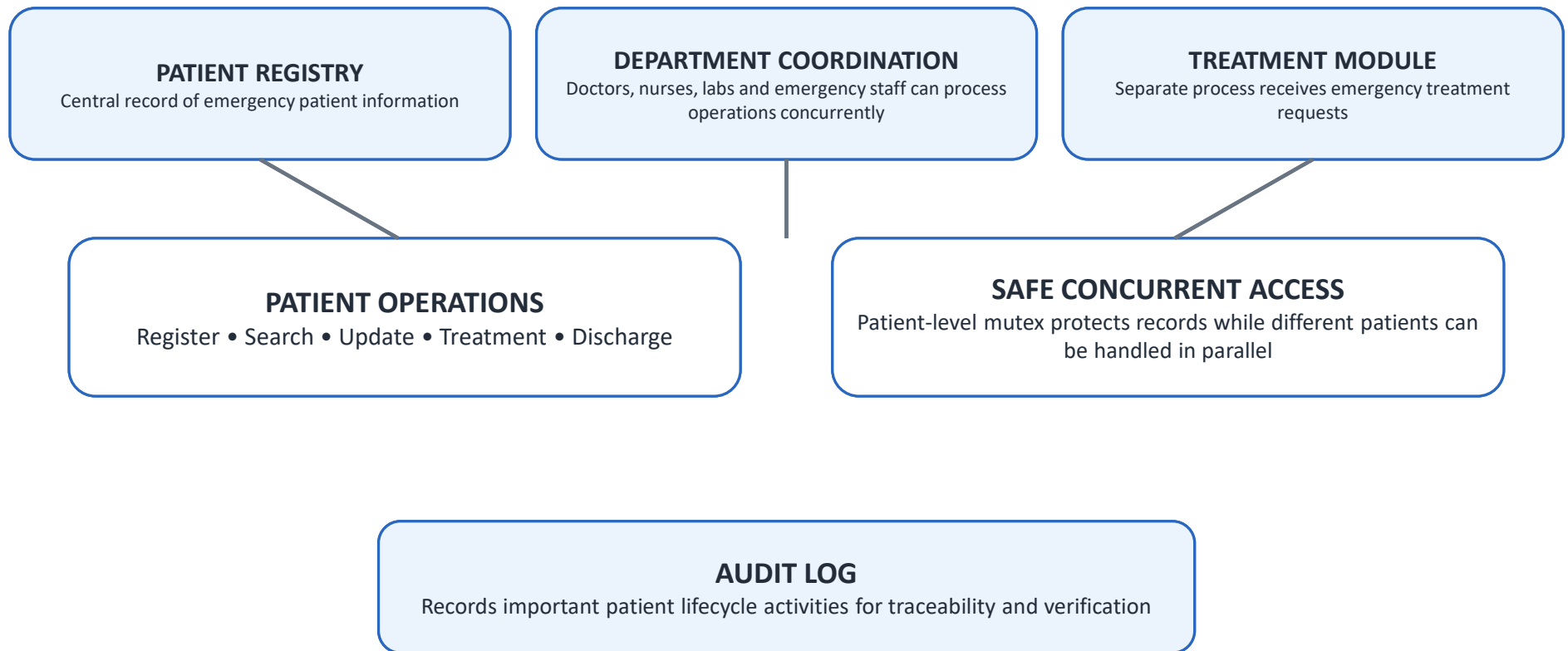
System Architecture & Emergency Coordination Prototype

ARCHITECTURE OVERVIEW

System Core • Problem • Architecture • Critical Conditions • Solution • End-to-End Flow

1. System Core

What the system manages and coordinates



2. Problem Layer

Challenges the architecture is designed to address

1. CONCURRENT ACCESS

Multiple departments may access the same patient record at the same time.

2. COARSE LOCKING

A single global lock can block unrelated patient operations and reduce scalability.

3. UNSYNCHRONIZED UPDATES

Concurrent read–modify–write operations may overwrite another department's update.

4. CRITICAL CASES

Emergency requests need priority handling so urgent cases are processed first.

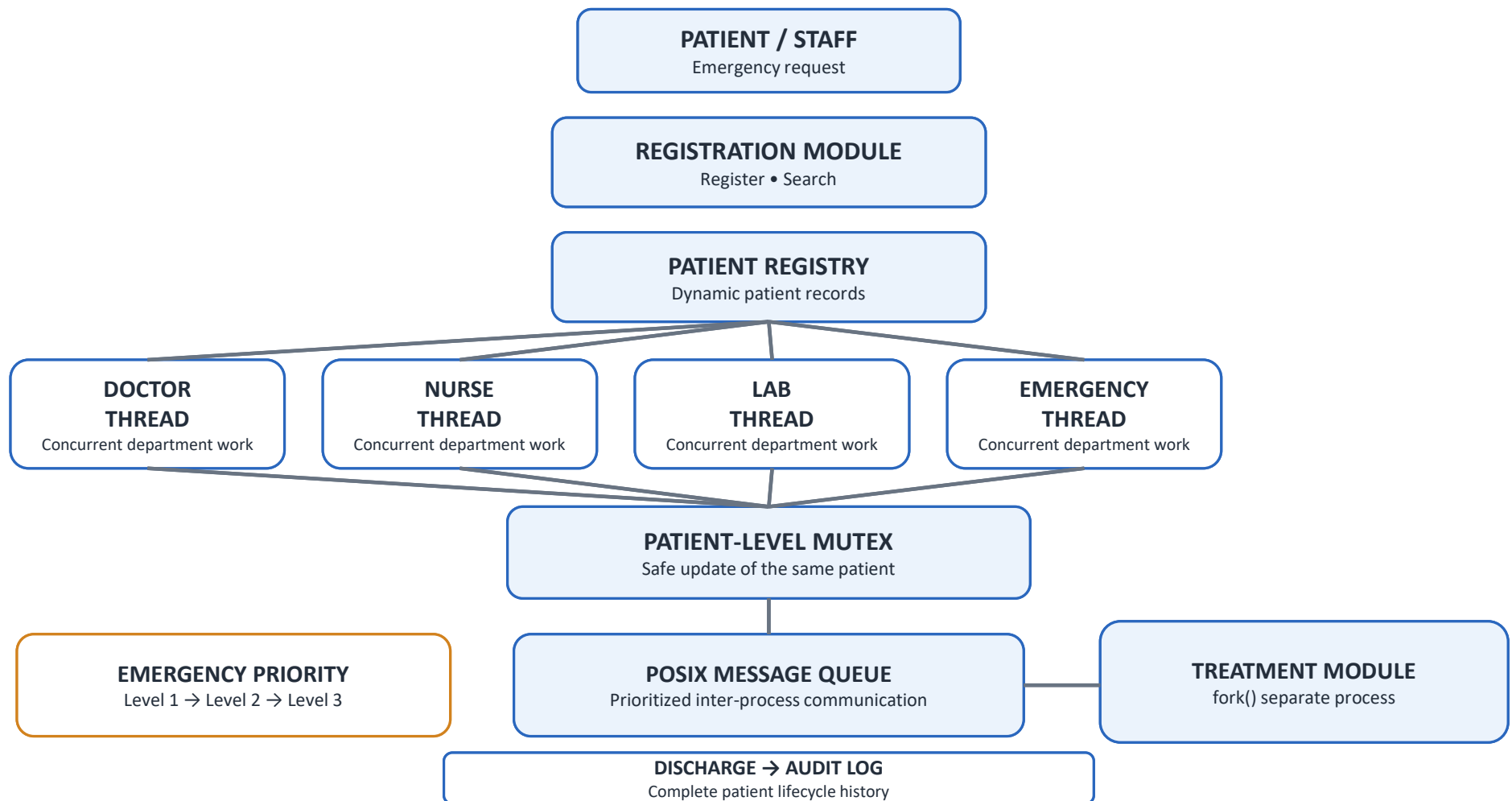
5. TRACEABILITY

Registration, updates and discharge need an activity history for accountability and verification.

IMPACT

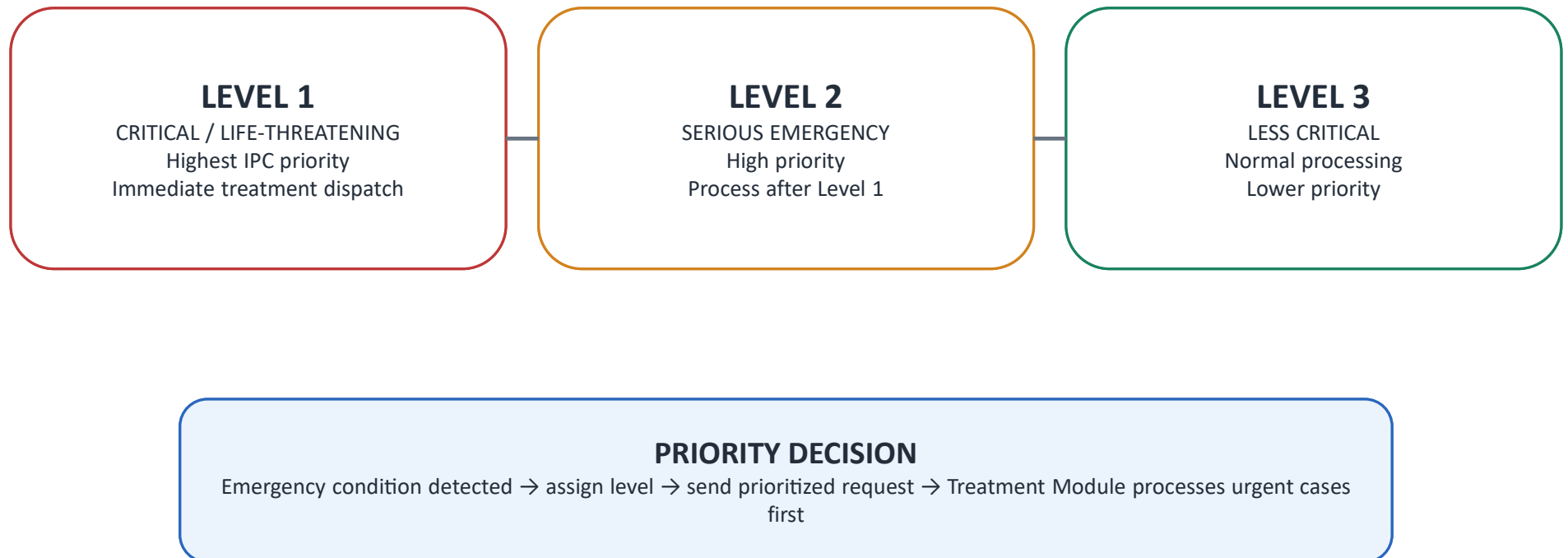
Waiting • Conflicting updates • Inconsistent records • Delayed emergency handling • Limited traceability

3. Complete System Architecture Diagram



4. Critical Conditions

Priority-based emergency treatment handling



5. Solution Layer

Design choices that address the identified problems

FINE-GRAINED SYNCHRONIZATION

One independent mutex for each patient record.

PARALLEL PROCESSING

Different patient records can be processed concurrently.

SAFE SERIALIZATION

Updates to the same patient are serialized safely.

SEPARATE TREATMENT PROCESS

fork() creates a separate Treatment Module.

PRIORITY IPC

POSIX message queues carry prioritized emergency requests.

AUDITABILITY

File-based history records important patient lifecycle events.

RESULT

Safe + consistent updates • Faster parallel work • Priority treatment • Traceable operations

6. End-to-End System Flow

Complete emergency patient lifecycle



FINAL SYSTEM OUTCOME

A coordinated emergency workflow that protects patient records, prioritizes critical requests, supports concurrent department work, and maintains traceability.